

Project in sw-hw co-design course

Musa Eisa – 212483341- musaeisa9@campus.technion.ac.il

Saadi Saadi – 212484471 – saadisaadi1@campus.technion

Github link:

https://github.com/musaeisa9/sw-hw_co-design_project

Raytrace Benchmark

Analysis and Software Optimization

HWSW Project | pyperformance benchmark

Key result: The optimized code reduced the benchmark time from 2.17 s to 1.55 s. This is a 28.6% lower runtime, or 1.40x speedup.

1. Benchmark Overview

The Raytrace benchmark renders a small 3D scene in Python. For each pixel, it creates a ray from the camera, checks where the ray hits the scene, chooses the closest hit, calculates lighting and shadows, and creates reflection rays when needed.

Workload. The default image is 100 x 100 pixels, so the renderer starts with 10,000 camera rays. The scene has seven spheres, one floor plane, so ray-object intersection and vector math are repeated many times.

Main libraries. The benchmark uses Python's math module for operations such as square root and trigonometry, array for the image buffer, and pyperformance for running and timing the benchmark.

Main data structures. The code uses classes such as Vector, Point, Ray, Sphere, Halfspace, Scene, Canvas, and Surface. Scene objects are stored as pairs of geometry and surface information. During rendering, the original code also creates many temporary objects, lists, and tuples.

2. Profiling Methodology

The benchmark was profiled in the course QEMU/Linux environment using Linux perf. The default hardware event did not give useful samples in the VM, so we used the cpu-clock flag. We also used python3-dbg so perf could show CPython function names.

```
perf record -e cpu-clock -F 999 -g -o raytrace.data -- \
python3-dbg -m pyperformance run --bench raytrace
perf report -i raytrace.data --stdio > raytrace_report.txt
perf script -i raytrace.data > raytrace.perf
stackcollapse-perf.pl raytrace.perf > raytrace.folded
flamegraph.pl raytrace.folded > raytrace.svg
```

We repeated the same profiling steps after the code changes. We used the pyperformance result to compare total execution time, and we generated a new perf report and flame graph for the optimized version.

Important note. The perf trace includes both the Raytrace code and extra work from CPython and pyperformance. We did not try to optimize interpreter, Unicode, or JSON functions. Instead, we focused on memory-allocation functions that matched temporary objects created by the Raytrace code.

3. Baseline Profiling and Bottleneck Identification

Baseline benchmark time: 2.17 s

The baseline perf report showed many samples in Python memory allocation, memory initialization, and object destruction. Important functions included:

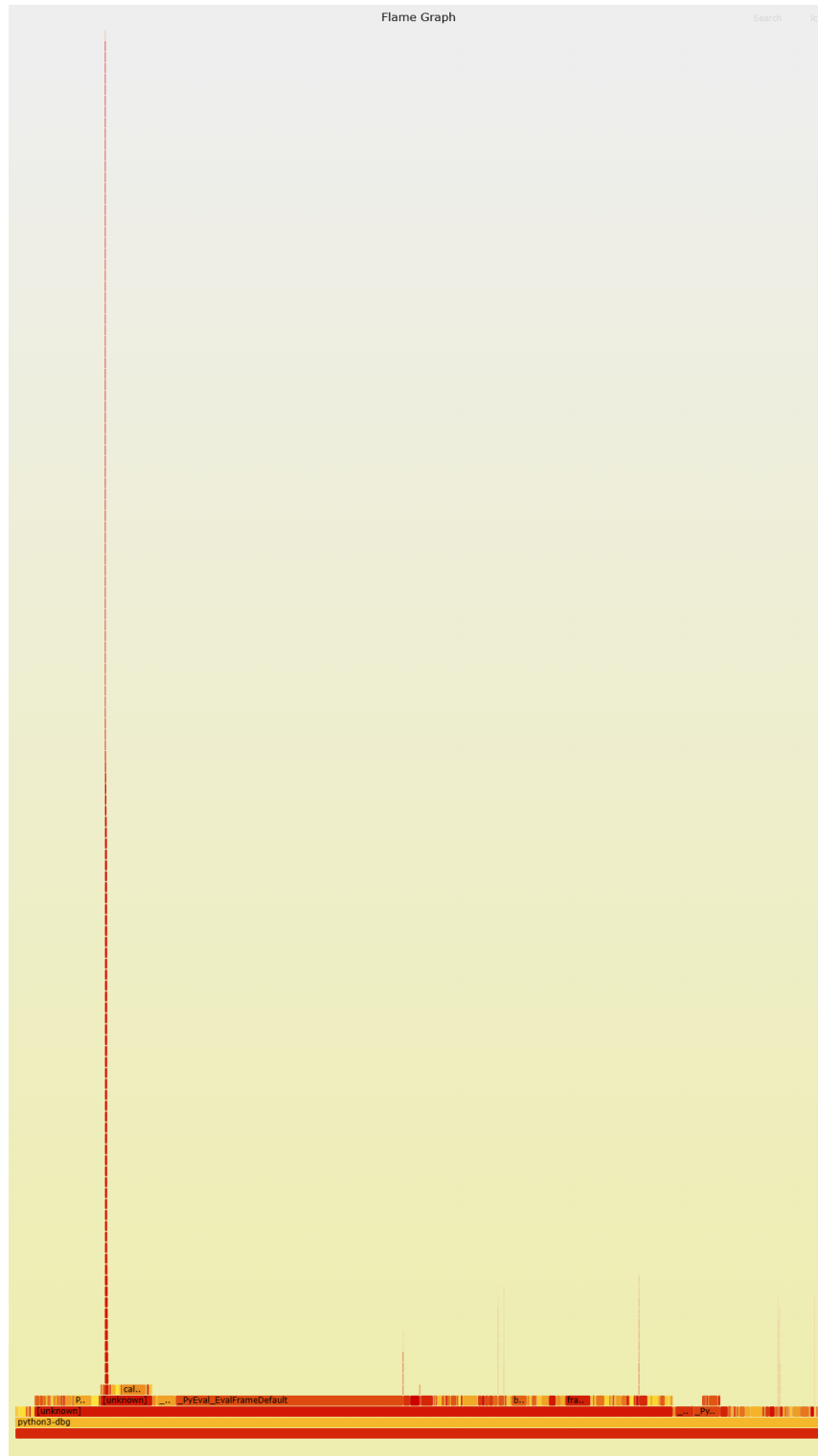
C-level routine	Baseline overhead	Meaning
_PyMem_DebugCheckAddress	3.48%	Debug memory/address checking
read_size_t	2.49%	Debug allocator metadata
__memset_avx2_unaligned_erms	2.95%	Low-level memory filling
_PyObject_Malloc	2.03%	Python object allocation
pymalloc_pool_extend	1.55%	Python memory-pool growth
tupledealloc	1.11%	Tuple destruction
_PyMem_DebugRawAlloc	1.02%	Debug memory allocation
frame_dealloc	0.82%	Python frame destruction
_PyObject_Free	0.75%	Python object deallocation

What this means. The problem is not one single memory function. The profile shows the same pattern across allocation, memory filling, and object destruction. Together, these functions take a large part of the trace. We use this as evidence that temporary Python objects are causing important overhead.

Other runtime activity. `_PyEval_EvalFrameDefault` (~8.05%) is the main CPython bytecode loop. Unicode and JSON functions come from Python/pyperformance work around the benchmark, not from ray intersections. We therefore did not choose them as optimization targets.

4. Baseline Flame Graph

The baseline flame graph gives a general view of where execution time is spent. Most of the samples are inside the CPython interpreter and its internal functions. To understand the bottleneck in more detail, we also examined the perf report and matched the reported functions with the Raytrace source code.



5. Root-Cause Analysis

Looking at the source code showed two main places that create many temporary objects.

5.1 Temporary Vector allocation in `Sphere.intersectionTime()`

In the original sphere-intersection code, `cp = centre - ray.point` creates a new temporary Vector object. The function then uses this vector for dot products and calculates the discriminant to decide whether the ray hits the sphere.

```

def intersectionTime(self, ray):
    cp = self.centre - ray.point
    v = cp.dot(ray.vector)
    discriminant = (self.radius * self.radius) - (cp.dot(cp) - v * v)
    if discriminant < 0:
        return None
    else:
        return v - math.sqrt(discriminant)

```

This function is called many times for camera rays, shadow rays, and reflection rays. Because of that, even one temporary Vector per test creates a lot of extra allocations.

5.2 Temporary list and tuple creation in Scene.rayColour()

The original rayColour() function builds a new intersections list for every ray. The list contains one tuple (object, t, surface) for every scene object. The code then scans the list to find the closest hit and throws the list away. This creates many short-lived lists and tuples.

```

def rayColour(self, ray):
    if self.recursionDepth > 3:
        return (0, 0, 0)
    try:
        self.recursionDepth = self.recursionDepth + 1
        intersections = [(o, o.intersectionTime(ray), s)
                        for (o, s) in self.objects]
        i = firstIntersection(intersections)
        if i is None:
            return (0, 0, 0) # the background colour
        else:
            (o, t, s) = i
            p = ray.pointAtTime(t)
            return s.colourAt(self, ray, p, o.normalAt(p))
    finally:
        self.recursionDepth = self.recursionDepth - 1

```

6. Software Optimizations

We optimized these two places while keeping the same raytracing equations and behavior.

6.1 Optimization A - flatten Sphere.intersectionTime() arithmetic

Instead of creating cp as a Vector and then calling Vector.dot(), the optimized code reads the x, y, and z values directly and performs the same math with scalar expressions. This removes the temporary Vector object and several method calls from every sphere-intersection test.

```

def intersectionTime(self, ray):
    cpx = self.centre.x - ray.point.x
    cpy = self.centre.y - ray.point.y
    cpz = self.centre.z - ray.point.z

    v = (cpx * ray.vector.x) + (cpy * ray.vector.y) + (cpz * ray.vector.z)
    cp_dot_cp = (cpx * cpx) + (cpy * cpy) + (cpz * cpz)

    discriminant = (self.radius * self.radius) - (cp_dot_cp - v * v)
    if discriminant < 0:
        return None
    else:
        return v - math.sqrt(discriminant)

```

This change does not make Python floating-point operations free. CPython still handles the float calculations. The improvement comes from creating fewer Vector objects and making fewer Python method calls.

6.2 Optimization B - search for the nearest hit in place

The optimized rayColour() function no longer creates an intersections list. It checks the objects one by one and keeps the closest valid hit found so far. This removes the list and most of the temporary tuples. A tuple is still created when a new closest hit is found, but this happens much less often.

```

def rayColour(self, ray):
    if self.recursionDepth > 3:
        return (0, 0, 0)
    try:
        self.recursionDepth += 1

        # Find the closest intersection without building lists/tuples
        closest_i = None
        min_t = float('inf')

        for o, s in self.objects:
            t = o.intersectionTime(ray)
            if t is not None and t > -EPSILON and t < min_t:
                min_t = t
                closest_i = (o, t, s)

        if closest_i is None:
            return (0, 0, 0)
        else:
            o, t, s = closest_i
            p = ray.pointAtTime(t)
            return s.colourAt(self, ray, p, o.normalAt(p))
    finally:
        self.recursionDepth = self.recursionDepth - 1

```

Together, the two changes reduce temporary objects both inside each sphere test and around the loop that checks all objects.

7. Performance Results and Validation

We ran the optimized benchmark in the same environment using the same method. The execution time dropped from 2.17 s to 1.55 s.

Metric	Baseline	Optimized	Observed change
Execution time	2.17 s	1.55 s	28.6% lower runtime; 1.40x speedup
pymalloc_pool_extend	1.55%	0.10%	93.5% reduction
_PyMem_DebugCheckAddress	3.48%	1.49%	57.2% reduction
_PyObject_Malloc	2.03%	0.88%	56.7% reduction
_PyMem_DebugRawAlloc	1.02%	0.43%	57.8% reduction
tupleddealloc	1.11%	0.67%	39.6% reduction
_PyObject_Free	0.75%	0.30%	60.0% reduction

The runtime was reduced by $(2.17 - 1.55) / 2.17 = 28.6\%$. This is also a 1.40x speedup. The result is above the project's required 7% improvement.

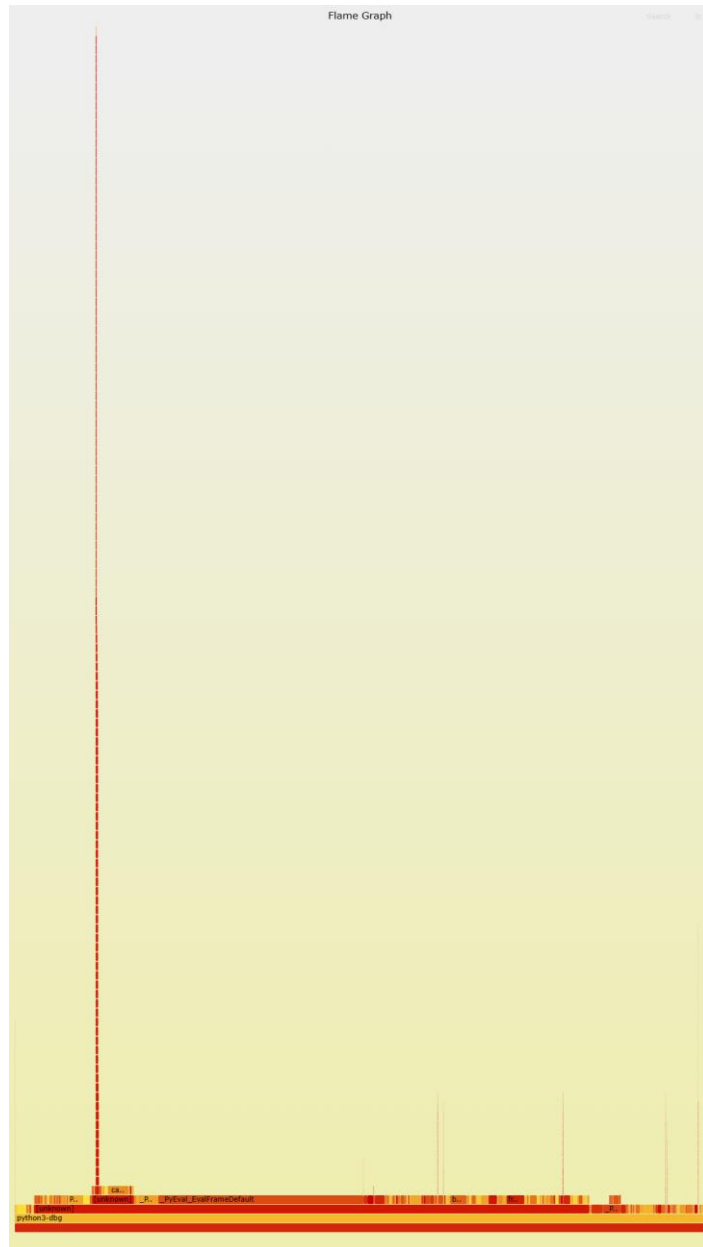
8. Post-Optimization Profile

After the optimization, we generated another flame graph and compared it with the original one. The two flame graphs looked very similar, so the improvement was not easy to see from the flame graphs alone. Because of this, we used the perf reports for the detailed before-and-after comparison. The perf report made the reduction in memory-management functions much clearer. For example, `_PyObject_Malloc`, `_PyMem_DebugCheckAddress`, and other allocation/deallocation functions took a smaller part of the optimized profile. Arithmetic also became easier to see; for example, `float_mul` appears at 1.25% in the optimized trace. This supports our conclusion that the changes reduced memory-allocation overhead.

```

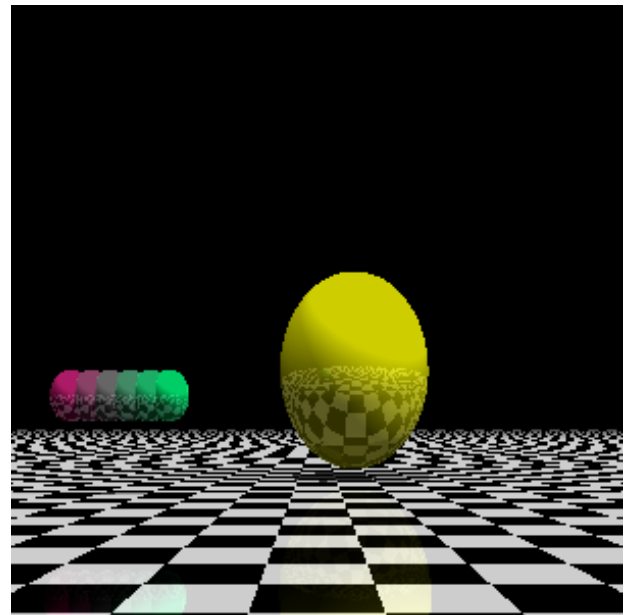
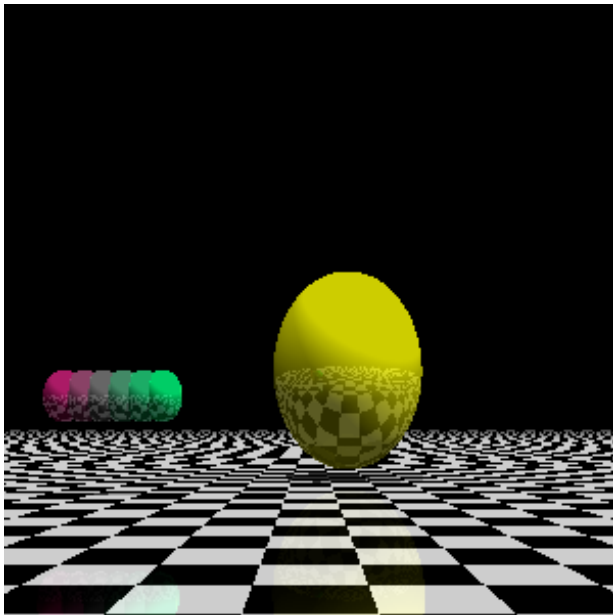
1.25%      1.25% python3-dbg python3.10d          [...] float_mul.lto_priv.0
|
--1.18%--0x9ad3c0
|
float_mul.lto_priv.0

```



9.comparing images

```
root@ubuntu:~/project# cmp original.ppm optimized.ppm
root@ubuntu:~/project#
```



10. Conclusion

The perf results showed a large amount of Python object-management work during rendering. Looking at the source code showed two important causes: a temporary Vector in Sphere.intersectionTime() and a temporary intersections list with tuples in Scene.rayColour().

Replacing the temporary Vector with direct coordinate math and finding the closest hit directly in the object loop reduced the time from 2.17 s to 1.55 s. This is a 28.6% lower runtime, or 1.40x speedup. Memory-allocation functions also became much smaller in the profile. The optimized code is now a better starting point for a hardware accelerator for repeated ray-object intersection work.

Pyflate Benchmark

Analysis and Software Optimization

HWSW Project | pyperformance benchmark

Key result: The optimized code reduced the benchmark time from 3.18 s to 2.78 s. This is a 12.6% lower runtime, or about 1.14x speedup.

1. Benchmark Overview

The Pyflate benchmark is a pure-Python decompression benchmark. In this project, the benchmark processes a Bzip2-compressed file and reconstructs the original data using Python code.

Processing flow. The decoder reads the compressed bit stream, builds Huffman tables, decodes Huffman symbols, applies Move-To-Front (MTF) decoding, reverses the Burrows-Wheeler Transform (BWT), and then finishes the Bzip2 run-length decoding.

Main data structures. The code uses bit-field objects for reading individual bits, Huffman tables for variable-length codes, Python lists for selectors and the MTF symbol list, and byte buffers for intermediate and final decompressed data.

2. Profiling Methodology

The benchmark was profiled in the course QEMU/Linux environment using Linux perf. As with Raytrace, we used the cpu-clock event because the default hardware event did not give useful samples in the VM. We also used python3-dbg so perf could show CPython function names.

```
perf record -e cpu-clock -F 999 -g -o pyflate.data -- \
python3-dbg -m pyperformance run --bench pyflate
perf report -i pyflate.data --stdio > pyflate_report.txt
perf script -i pyflate.data > pyflate.perf
stackcollapse-perf.pl pyflate.perf > pyflate.folded
flamegraph.pl pyflate.folded > pyflate.svg
```

We repeated the same profiling steps after changing the MTF implementation. We used the pyperformance result to compare total execution time and the perf reports to compare the important C-level list and memory functions before and after the change.

3. Baseline Profiling and Bottleneck Identification

Baseline benchmark time: 3.18 s

The baseline perf report showed a clear group of list and memory-management functions. These functions match operations used by the original Move-To-Front implementation.

C-level routine	Baseline overhead	Meaning
list_dealloc	2.48%	Destroying temporary Python lists
list_ass_slice	2.08%	Slice assignment on a Python list
list_concat	1.11%	Creating concatenated temporary lists

list_slice	1.05%	Creating list slices
__memset_avx2_unaligned_erms	3.02%	Low-level memory filling / cleanup work

What this means. The important point is the pattern: slicing, concatenation, slice assignment, and list destruction all appear in the profile. The original MTF helper performs exactly these kinds of operations, so it became the main software optimization target.

4. Baseline Flame Graph

The baseline flame graph gives a general view of the benchmark. Much of the graph is inside the Python interpreter, while the perf report gives clearer names and percentages for the list operations that we wanted to optimize.



5. Root-Cause Analysis

The profile was matched with the Pyflate source code. The main target was the Move-To-Front helper used during Bzip2 decoding.

5.1 Original Move-To-Front implementation

The MTF list keeps recently used symbols near the front. When the decoder uses the symbol at position *c*, that symbol must be moved to index 0 while the earlier symbols shift one position to the right.

```
def move_to_front(l, c):
    l[:] = l[c:c + 1] + l[0:c] + l[c + 1:]
```

This version creates several temporary lists. The slices create new list objects, the + operations create concatenated lists, and the final slice assignment copies the result back into *l*. Because MTF is called repeatedly during decoding, these temporary allocations and copies add significant overhead.

6. Software Optimization

6.1 Update the same list in place

The optimized version keeps the same MTF behavior but avoids building several temporary lists. It removes the selected value from its current position and inserts it at the front of the same list.

```
def move_to_front(l, c):
    val = l.pop(c)
    l.insert(0, val)
```

The list still has to move some elements internally, so the operation is not free. The improvement comes from avoiding repeated Python slicing and concatenation and from keeping the update on the same list object.

7. Performance Results and Validation

We ran the optimized benchmark in the same environment using the same method. The execution time dropped from 3.18 s to 2.78 s.

```
pyflate: Mean +- std dev: 3.18 sec +- 0.03 sec
```

```
.....
pyflate: Mean +- std dev: 2.78 sec +- 0.03 sec
```

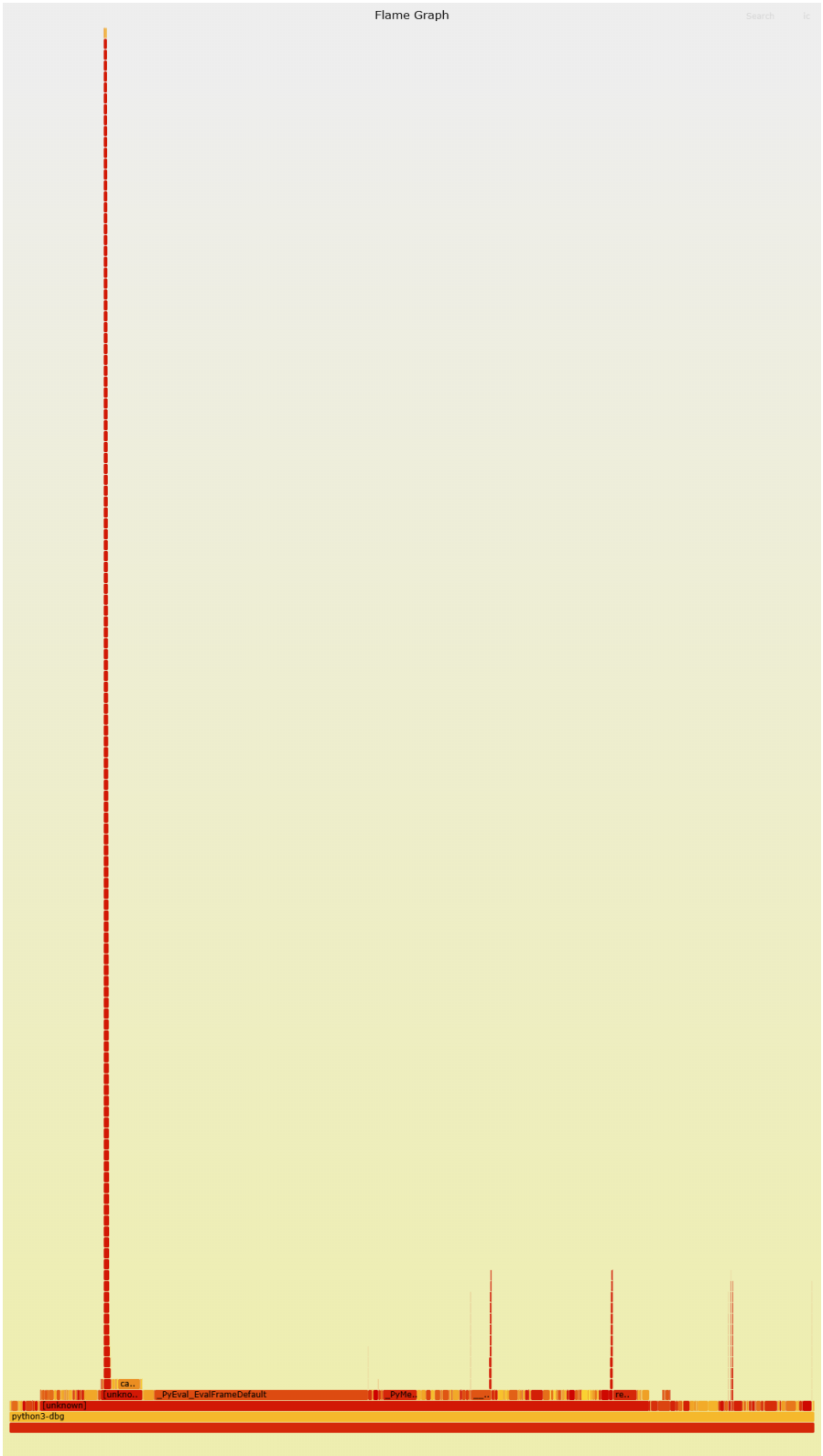
Metric	Baseline	Optimized	Observed change
Execution time	3.18 s	2.78 s	12.6% lower runtime; 1.14x speedup
list_dealloc	2.48%	0.31%	87.5% reduction
list_ass_slice	2.08%	0.06%	97.1% reduction
list_concat	1.11%	0.00%	Eliminated from the shown profile
list_slice	1.05%	0.00%	Eliminated from the shown profile
__memset_avx2...	3.02%	2.52%	16.5% reduction

The runtime reduction is $(3.18 - 2.78) / 3.18 = 12.6\%$. This is about a 1.14x speedup and is above the project requirement of 7%.

8. Post-Optimization Profile

After the optimization, we generated another flame graph and another perf report. The two flame graphs still looked very similar at a high level, so the perf report was more useful for the detailed before-and-after comparison. The report showed that `list_dealloc`, `list_ass_slice`, `list_concat`, and `list_slice` became much

smaller or disappeared from the shown top functions. This matches the expected effect of removing slicing and concatenation from `move_to_front()`.



9. Conclusion

The baseline profiling showed repeated Python list slicing, concatenation, slice assignment, list destruction, and related memory work. Looking at the source code connected these costs to the Move-To-Front helper inside the Bzip2 decoding path.

Replacing the slicing-based MTF update with `pop()` and `insert()` kept the same behavior while avoiding several temporary lists. The benchmark time dropped from 3.18 s to 2.78 s, which is a 12.6% lower runtime, or about a 1.14x speedup. The perf report also showed large reductions in the list functions targeted by the change.

This optimized version provides a cleaner software baseline for the next stage of the project, including possible hardware acceleration of the MTF-related decoding work.

Accelerating Huffman decoding phase in pyflate benchmark

1. Profiling Analysis and Acceleration Justification The decision to accelerate the Huffman decoding phase was driven by a combination of application-level (cProfile) and OS-level (perf) profiling data. Following the software optimization of the Move-To-Front (MTF) algorithm—which eliminated the object churn associated with array slicing—the remaining execution bottleneck was isolated to pure algorithmic and bit-manipulation overhead.

The Software Bottleneck ($O(N)$ Sequential Search): The cProfile trace revealed that functions responsible for bit-level extraction and symbol matching (`find_next_symbol`, `snoopbits`, `readbits`, `_mask`) consumed approximately 75% of the total execution time. In the baseline Python implementation, `find_next_symbol` relies on an $O(N)$ sequential loop. For every symbol in the compressed stream, the CPU iterates through a list of up to 258 `HuffmanLength` objects, extracting variable-length bits and comparing them one-by-one. Furthermore, extracting these non-byte-aligned bits requires heavy interpreter overhead, resulting in hundreds of cycles per symbol.

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.493	0.493	2.041	2.041	run_benchmark.py:395(decode_huffman_block)
148271	0.325	0.000	1.054	0.000	run_benchmark.py:224(find_next_symbol)
341601	0.224	0.000	0.492	0.000	run_benchmark.py:112(snoopbits)
156708	0.192	0.000	0.254	0.000	run_benchmark.py:117(readbits)
655017	0.129	0.000	0.129	0.000	run_benchmark.py:54(_mask)
1	0.111	0.111	0.245	0.245	run_benchmark.py:307(bwt_reverse)
1	0.078	0.078	0.104	0.104	run_benchmark.py:293(bwt_transform)

The Hardware Advantage: By exploiting the mathematical properties of Canonical Huffman codes—where codes of the same bit-length are assigned sequential binary values—the hardware can determine the symbol in $O(1)$ time without a massive LUT. A custom RTL datapath using 20 parallel limit comparators completely bypasses the $O(N)$ search and executes the variable-length bit shifting with zero clock-cycle overhead, decoding one symbol per cycle.

2. Hardware Micro-Architecture (The Core Datapath) The core algorithm is implemented as a 4-stage, fully pipelined datapath. It relies on four dynamically configured arrays per Huffman table: `min_code`, `max_code`, `base_ptr`, and the actual `symbol_sram`.

Stage 1: Sliding Window and Parallel Compare The datapath ingests 32-bit chunks into a 64-bit sliding window buffer. The top 20 valid bits ("peek bits") are broadcast simultaneously to 20 parallel comparators. These comparators evaluate the peek bits against the active table's limit registers:

$$\text{min_code}[L] \leq (\text{peek_bits} \gg (20 - L)) \leq \text{max_code}[L]$$

This combinational logic generates a one-hot vector indicating the exact length L of the current symbol. A critical combinational feedback loop routes L back to the sliding window, shifting the buffer by L bits on the next clock edge. This guarantees a throughput of one symbol decoded per clock cycle without pipeline stalls.

To illustrate this parallel evaluation, consider the following example: Assume the active Huffman table is configured with the following limit registers, and the 64-bit sliding window buffer currently holds a stream starting with the bits 101110... at the Most Significant Bit (MSB, bit 63).

- Length 2 limits: min = 00 (0), max = 01 (1)
- Length 3 limits: min = 100 (4), max = 101 (5)
- Length 4 limits: min = 1100 (12), max = 1100 (12)

In hardware, extracting the top L bits from the 64-bit window translates to a simple wire slice: `buffer[63 : 63-L+1]`. On a single clock cycle, all 20 comparators extract their respective wire slices and evaluate them simultaneously:

- Comparator for $L=2$: Extracts buffer, which is 10 (Decimal 2). Condition: Is $0 \leq 2 \leq 1$? $\rightarrow$ False
- Comparator for $L=3$: Extracts buffer, which is 101 (Decimal 5). Condition: Is $4 \leq 5 \leq 5$? $\rightarrow$ True (Match)
- Comparator for $L=4$: Extracts buffer, which is 1011 (Decimal 11). Condition: Is $12 \leq 11 \leq 12$? $\rightarrow$ False

Because the $L=3$ comparator resolves to true, the hardware's one-hot vector immediately asserts at index 3. The datapath uses this one-hot signal to route the length-3 base pointer (`base_ptr[3]`) and minimum code (`min_code[3]`) to Stage 2 for the ALU calculation. Simultaneously, it feeds $L=3$ back to the sliding window, instructing it to shift the buffer left by 3 bits on the next rising clock edge so the next symbol's bits align precisely at bit 63.

Stage 2: ALU Address Calculation The one-hot vector multiplexes the matching L 's `base_ptr` and `min_code`. The ALU calculates the exact SRAM index using a single subtraction and addition:

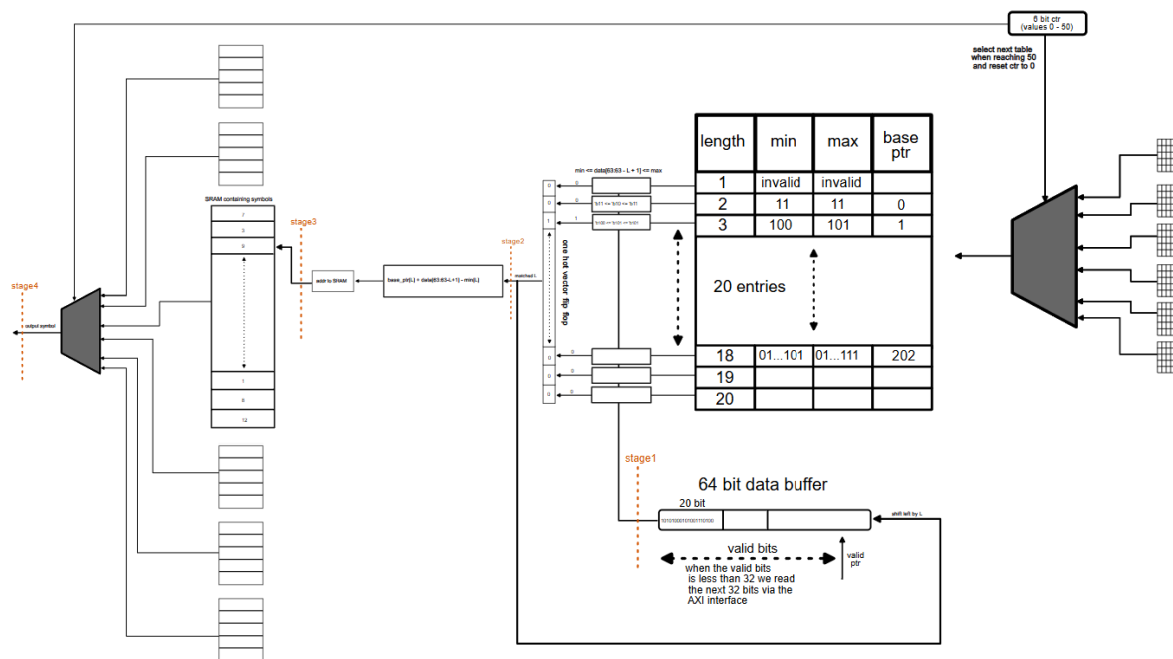
$$Index = base_ptr[L] + (Code - min_code[L])$$

The resulting index is flopped into the address port of the Symbol SRAMs.

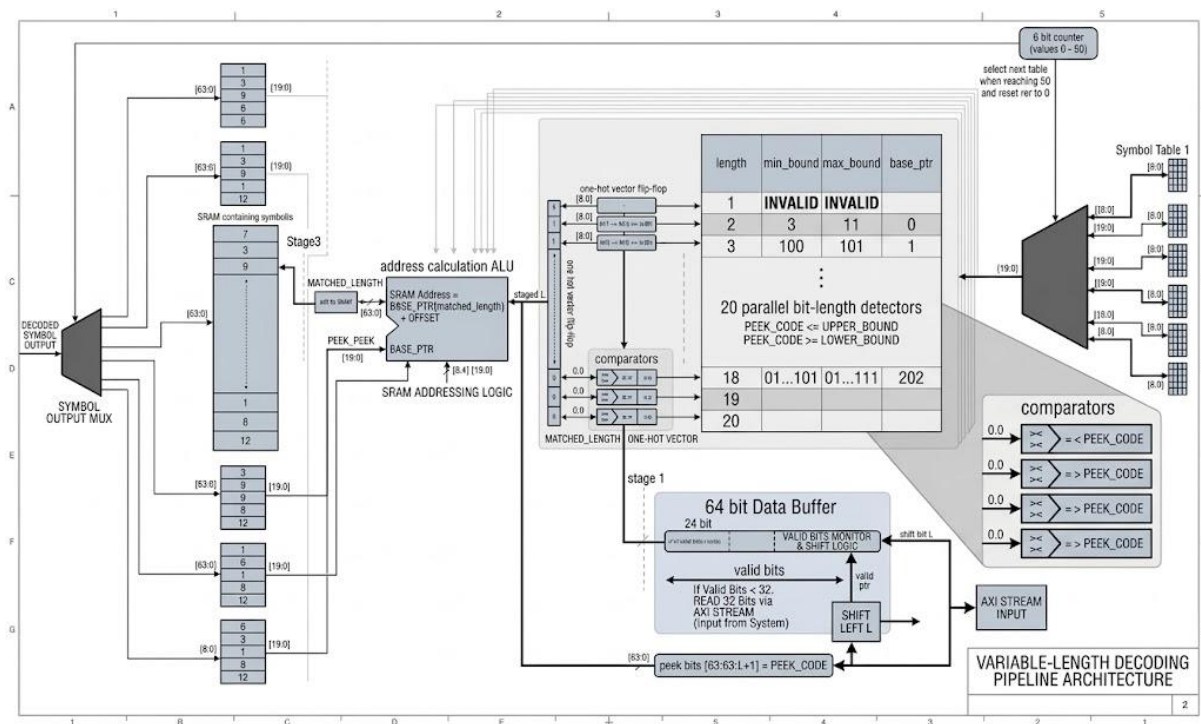
Stage 3 & 4: Memory Fetch and Output The calculated address is routed to the active table's on-chip SRAM. Because on-chip SRAM has a deterministic 1-cycle latency, the decoded 9-bit symbol (0 to 258) is retrieved on the next clock edge, zero-padded to 16 bits, and pushed to the output register.

Table Switching Logic: Bzip2 dictates that the active Huffman table changes exactly every 50 symbols. A 6-bit hardware counter tracks decoded symbols. A parallel Selector SRAM pre-fetches the next table index. When the counter reaches 50, the core instantaneously swaps the active min/max/base registers and SRAM read ports, preventing any pipeline bubbles during table transitions.

Block diagram:



We requested from Gemini banana to draw a diagram similar to the one we drew above to be clearer:



3. System Integration and Interfaces (The Wrapper) The Huffman decoder is connected to the SoC through the huffman_accelerator_top wrapper. The wrapper connects the decoder core to two main interfaces: AXI-Lite for configuration and AXI-Stream for the compressed input and decoded output. The overall system is shown in the block diagram above.

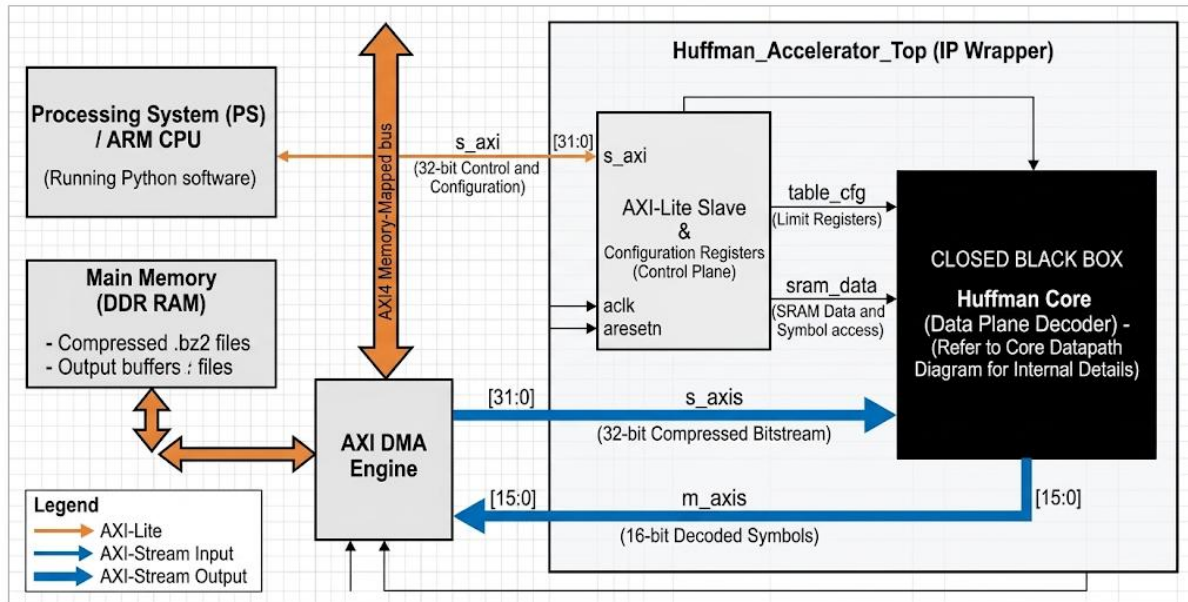
Control Interface The CPU configures the accelerator through a 32-bit AXI-Lite interface. The AXI-Lite module receives software writes and stores the Huffman configuration in the hardware. For each Huffman table, the software provides:

- min_code[L]
- max_code[L]
- base_idx[L] for code lengths from 1 to 20.

The software also writes the selector information and the symbol tables used by the decoder. A start register is used to begin the decoding operation. The limit values are stored in registers so that they can be accessed in parallel by the 20 comparators in the decoder. The selector and symbol data are stored in dual-port RAMs.

Data Interface The compressed Bzip2 data is transferred to the accelerator through a 32-bit AXI-Stream input. The Huffman core processes the incoming bitstream and

produces decoded symbols through a 16-bit AXI-Stream output. An AXI DMA engine is used between DDR memory and the accelerator. The TX side of the DMA sends the compressed bitstream to the accelerator, while the RX side receives the decoded symbols and stores them back in memory. This allows the CPU to avoid copying the whole compressed stream and decoded result word by word.



System Integration of Huffman Decoding Accelerator via SoC AXI and DMA Interconnects.

4. Hardware/Software Co-Design The software still performs the parts of Bzip2 that are not implemented in hardware. Its main role is to prepare the Huffman tables, configure the accelerator, start the DMA transfer, and process the decoded symbols returned by the hardware.

Hardware Configuration Conceptually, the Python code uses a small driver to access the accelerator's memory-mapped AXI-Lite address space. The driver maps the hardware address range and provides a simple `write_reg()` function for writing 32-bit values. The configuration process is:

1. The software generates the canonical Huffman information from the current Bzip2 block.
2. It writes the selector list to the selector memory.
3. For each Huffman table, it writes the `min_code`, `max_code`, and `base_idx` arrays.
4. It writes the corresponding symbol table to the symbol memory.
5. It writes to the START register to begin decoding.

This replaces the sequential Huffman search performed by the original Python implementation with the hardware decoder. The driver code used for the project is

conceptual and describes how the software would communicate with the RTL in a real SoC implementation. The exact DMA API depends on the target platform.

DMA Execution After the hardware is configured, the remaining compressed bytes are sent to the accelerator using DMA. The conceptual software flow is: Configure Huffman tables → Write START register → Send compressed data using TX DMA → Huffman decoder → Receive decoded symbols using RX DMA → Continue Bzip2 processing in software.

The hardware therefore replaces only the Huffman decoding loop rather than the entire Bzip2 decoder.

Software Processing After the Accelerator The accelerator returns the decoded Huffman values as 16-bit integers. The software then continues with the remaining Bzip2 operations. In particular, the software handles:

- RUNA/RUNB run-length processing
- the Move-To-Front (MTF) updates
- the remaining Burrows-Wheeler Transform processing

For example, the returned hardware symbols are processed by a loop that reconstructs repeated symbols and updates the MTF list. The existing optimized software implementation can therefore still be used after the hardware decoder finishes. This division keeps the hardware focused on the computationally expensive Huffman decoding stage while leaving the more irregular parts of Bzip2 in software. The original project design similarly keeps the RUNA/RUNB and MTF processing in software to avoid making the RTL unnecessarily complex.

5. Performance, Area, and Power Trade-offs The main hardware design choice is to use parallel logic for Huffman code detection and SRAM for the larger symbol tables.

Performance The main expected performance improvement comes from the parallel comparison and pipelined datapath. The decoder has to determine which of the 20 possible Huffman code lengths matches the incoming bits. Instead of checking the lengths one by one as in software, the hardware checks all 20 lengths in parallel. For each length, the incoming bits are compared with the corresponding min_code and max_code. The result is a one-hot match signal, which identifies the length of the current Huffman code. This parallel operation is the main source of the expected speedup over the software implementation, replacing the sequential search with simultaneous evaluation. After the pipeline is filled, the architecture is designed to decode approximately one Huffman symbol per clock cycle. For example, a 250 MHz operating frequency would correspond to a theoretical throughput of about 250 million symbols per second.

Area and Power The parallel comparators and additional pipeline registers slightly increase the hardware area compared with a simple sequential decoder, but they allow several possible Huffman lengths to be checked at the same time. This design balances memory usage and comparison speed by intelligently partitioning data. The `min_code`, `max_code`, and `base_idx` values are stored in registers. This is useful because all 20 lengths may need to be examined during the same cycle, and keeping the values in registers allows the comparators to access them in parallel.

Conversely, the actual decoded symbol values are stored in SRAM. There are many more symbol entries than configuration values, so storing the whole symbol table in flip-flops would use unnecessary hardware resources. Since only one symbol is fetched for each decoded code, an SRAM-based implementation is highly area-efficient.

From a power perspective, the accelerator performs the decoding much faster than the original software loop, which can reduce the amount of time the CPU needs to spend executing the Huffman decoder. The accelerator therefore trades some additional hardware and switching activity for lower execution time and potentially lower total system energy.